AIMLCZG546 · SESSION 4 · COMPANION READER

# Quality Attributes & Architecture

From non-functional requirements to architectural patterns for ML systems

Session 4 · Read alongside the lecture slides · Every slide example worked out in full

**How to use this companion.** The slides give you the formulas. This reader gives you the *why*. Every slide concept is expanded into a plain-English explanation. Every slide example is worked out step by step. Colour-coded boxes guide you through each idea. **Blue** = the core idea in plain words. **Amber** = a real-life analogy, before any math. **Green** = a worked scenario with all steps shown. **Red** = the common mistake to avoid. **Purple** = the one sentence to remember. Read it alongside the slides, in order.

## 1 Case studies: why architecture matters

Two real stories set the scene. They show why quality attributes matter *before* you write a single line of code.

### 1.1 Flipkart Big Billion Day 2014 — what went wrong

Flipkart ran India's biggest ever sale. Three lakh (300,000) orders arrived in six hours. Then the website crashed. Cart items vanished. Money left customers' accounts, but orders were never placed. Customers were furious.

The root cause was simple: Flipkart had never written down what the system *must* do at peak load. There was no scalability target. There was no availability target. The architecture was never tested against those numbers.

### 1.2 Hotstar Cricket World Cup 2019 — what went right

India vs New Zealand in the semi-final. **25 million users** watched on Hotstar at the same time. Zero crashes. Zero buffering complaints.

Why? Hotstar's team had written down two clear targets years earlier: *"Handle 25 million concurrent streams"* and *"99.99% uptime during live matches."* Every architecture decision — microservices, CDN edge caching, auto-scaling Kubernetes clusters — was made to hit those two numbers.

**☞ The intuition**

Quality attributes are not afterthoughts. They are the *brief* you hand the architect. The architecture exists to meet the QAs. Without QAs, the architect has nothing to aim at — and the system fails under pressure.

**★ Everyday picture**

Think of building a bridge. You do not start pouring concrete. First you ask: how heavy are the trucks? How many at once? 50 tonnes? 1,000 trucks per day? Those numbers are the quality attributes. The answers drive everything — width, material, number of lanes. Architecture works the same way.

**✏ Worked example: Comparing the two architectures**

**Step 1.** **Write down Flipkart's situation.** No scalability QA was defined. The system used a single monolithic app and one central database.

**Step 2.** **Trace the failure.** 300,000 users hit at once. The single database became the bottleneck. Servers restarted under load and lost all session data (cart contents) from memory. No fallback existed.

**Step 3.** **Write down Hotstar's situation.** QA 1: handle 25M concurrent streams. QA 2: 99.99% uptime during live events.

**Step 4.** **Trace the success.** Hotstar used microservices (each team's job was small and independent). Video was served from CDN edge nodes close to each user. Kubernetes auto-scaled new servers within seconds of a traffic spike. Load tests ran months before the World Cup — against the exact numbers in the QAs.

**Step 5.** **Read the lesson.** Same type of event (a flash traffic spike). Opposite outcomes. The only difference: Hotstar defined its QAs first, then built to meet them.

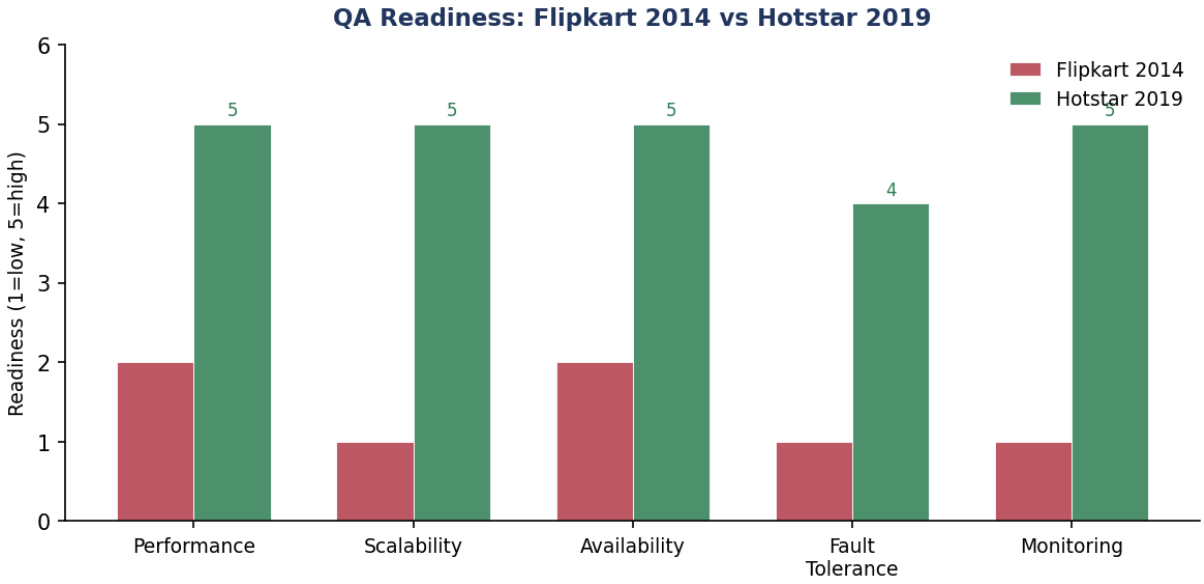


Figure 1: QA readiness scores for Flipkart 2014 versus Hotstar 2019. Higher bars mean the team had prepared for that attribute. Hotstar scored near-perfect across all five; Flipkart scored near-zero.

**✓ Key takeaway**

Define your quality attributes first. Build the architecture to meet them. Never do it the other way around.

**Where this shows up in ML.** The same lesson applies to ML systems. An ML model that scores 95% on a test set but takes 10 seconds per prediction fails in production if the QA says 200ms. Define the latency, throughput, and accuracy targets before choosing the model.

## 2 What is a quality attribute?

A **quality attribute (QA)**, also called a **non-functional requirement (NFR)**, says *how well* the system must do its job — not *what* it must do.

Formal definition (Len Bass, *Software Architecture in Practice*):

> "A quality attribute is a measurable or testable property of a system that is used to indicate how well the system satisfies the needs of its stakeholders."

☞ **The intuition**

A functional requirement says: *"The user can place an order."* A quality attribute says: *"The user can place an order in under 3 seconds, on a phone."* It adds: *"Even when 10 million others are ordering at the same time."* The second version is engineering-ready. The first is just a wish.

### 2.1 Quality attributes qualify functional requirements

QAs do not stand alone. Every QA attaches to a function. The same button click might carry three QA annotations:

- **Function:** "When the user presses the green button, the Options dialog appears."
- **Performance QA:** The dialog must appear within 200ms.
- **Availability QA:** This function must fail at most 0.05% of the time. When it fails, it must recover within 1s.
- **Usability QA:** At least 90% of new users must find the button without help in a usability test.

### 2.2 SMART quality attributes

A QA must be SMART to be useful:

- **S**pecific — well defined ("fast" is not specific; "200ms at the 95th percentile" is).
- **M**easurable — you can run a test and get a number.
- **A**ttainable — realistic with the tech and budget.
- **R**elevant — tied to a real stakeholder need.
- **T**ime-sensitive — measured under a specific load or time condition.

★ **Everyday picture**

"The app should be fast" is like a doctor saying "be healthier." A good doctor says "your resting heart rate should be below 70 bpm by 1 January." That is SMART: specific, measurable, attainable, relevant, time-bound. Write QAs the doctor way, not the vague way.

**✏ Worked example: SMART QAs for a food delivery app**

The lecture asks: list quality attributes for an app like Swiggy or Zomato. Here is how each vague wish becomes a SMART QA:

**Step 1.** **Performance.** Vague: "The app should be fast." SMART: Load restaurant menus in $\leq$2 s. Complete order placement in $\leq$3 s for 95% of requests.

**Step 2.** **Availability.** Vague: "The app should be up." SMART: 99.95% uptime per year. Always on, 24×7×365, except planned maintenance windows.

**Step 3.** **Usability.** Vague: "Easy to use." SMART: 90% of new users place their first order without help in a usability test.

**Step 4.** **Scalability.** Vague: "Handle peak traffic." SMART: Serve 10 million order requests per second at peak. Response time must not rise above 5 s.

**Step 5.** **Interoperability.** Vague: "Works with payment systems." SMART: 99% of transactions with Razorpay and Google Maps APIs must complete without error.

**Step 6.** **Reliability.** Vague: "Orders should be correct." SMART: 99.9% order accuracy — right food delivered to the right address.

Each step turns a wish into a test you can run. That is the point of SMART.

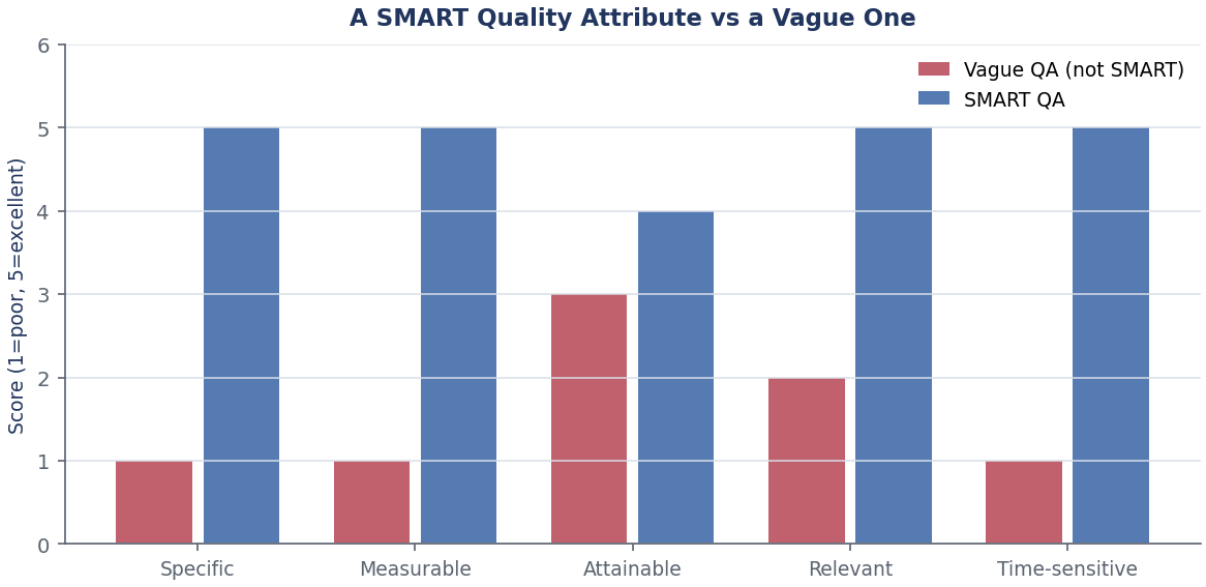


Figure 2: A SMART QA (blue) scores high on all five SMART dimensions. A vague QA (red) scores near zero on most. The gap shows exactly what "writing a good QA" means.

**✗ Watch out**

Do not write QAs *after* the architecture is built. That reverses the process. The architect needs the QAs to make decisions. If you hand them over after the decisions are made, the QAs cannot influence anything. Also, do not obsess over having many QAs. Fewer, clear, SMART QAs beat a long list of vague ones every time.

✓ Key takeaway

A QA says *how well*, not *what*. Write it SMART: specific, measurable, attainable, relevant, time-sensitive. A good QA is a test you can run.

**Where this shows up in ML.** In ML, QAs drive model selection. If the latency QA is 50ms, a large transformer is the wrong choice — pick a distilled or quantised model. If the accuracy QA is 99% precision on fraud, you cannot compromise it for speed. QAs force you to make those trade-offs explicit early.

## 3 Quality attributes specific to ML systems

ML systems have extra QAs not found in regular software. They arise because a model is a learned artifact that can drift, fail on new data, and produce unfair or unexplainable outputs.

Here are the main ML-specific QAs from the session:

**Accuracy** How well the model predicts. Measure with F1-score, AUROC, RMSE, or another task-specific metric.

**Scalability** Can the system handle more data, more users, or more calls without slowing down?

**Reliability** Does the system perform consistently under different conditions (time of day, data shifts, hardware faults)?

**Explainability** Can a human understand why the model made a specific prediction? Tools like SHAP or LIME help here.

**Robustness** Does the model still work when inputs are noisy, incomplete, or adversarial?

**Maintainability** How easy is it to update the model, retrain it, or fix a bug in the pipeline?

**Security & Privacy** Is the model protected from attacks? Is user data kept private?

**Fairness** Does the model give equally good results for all user groups?

**Data Drift Adaptability** Can the system detect when the input data has changed over time and adapt?

**Model Drift Monitoring** Does the system track when model performance drops after deployment?

**Reproducibility** Does the same training run produce the same model every time?

★ Everyday picture

**Robustness** is like a car that still handles correctly in rain, fog, and snow — not just on a sunny test track. A fraud detection model must work even when some transaction fields are missing (network timeout), or when a new type of attack appears. A self-driving vision system must still spot pedestrians in bad lighting.

**✏ Worked example: ML QA examples with targets**

**Step 1.** **Accuracy QA.** Target: F1-score $\geq 0.88$ on the held-out test set. Test: run the model on 10,000 labelled examples after training.

**Step 2.** **Robustness QA.** Target: accuracy drops by at most 3% when 10% of input features are missing. Test: randomly mask 10% of features; compare accuracy before and after.

**Step 3.** **Data Drift QA.** Target: raise an alert when the KL divergence between training and production distributions exceeds 0.1. Retrain within 24 hours of an alert.

**Step 4.** **Fairness QA.** Target: recall gap between demographic groups must be $\leq$2%. Test: compute recall for each group; check the largest gap.

**Step 5.** **Reproducibility QA.** Target: given the same data and random seed, model weights must differ by less than 0.001. Test: run training twice; compare weight tensors.

Notice each QA has a *target number* and a *test you can run.* That is what makes them engineering-ready.

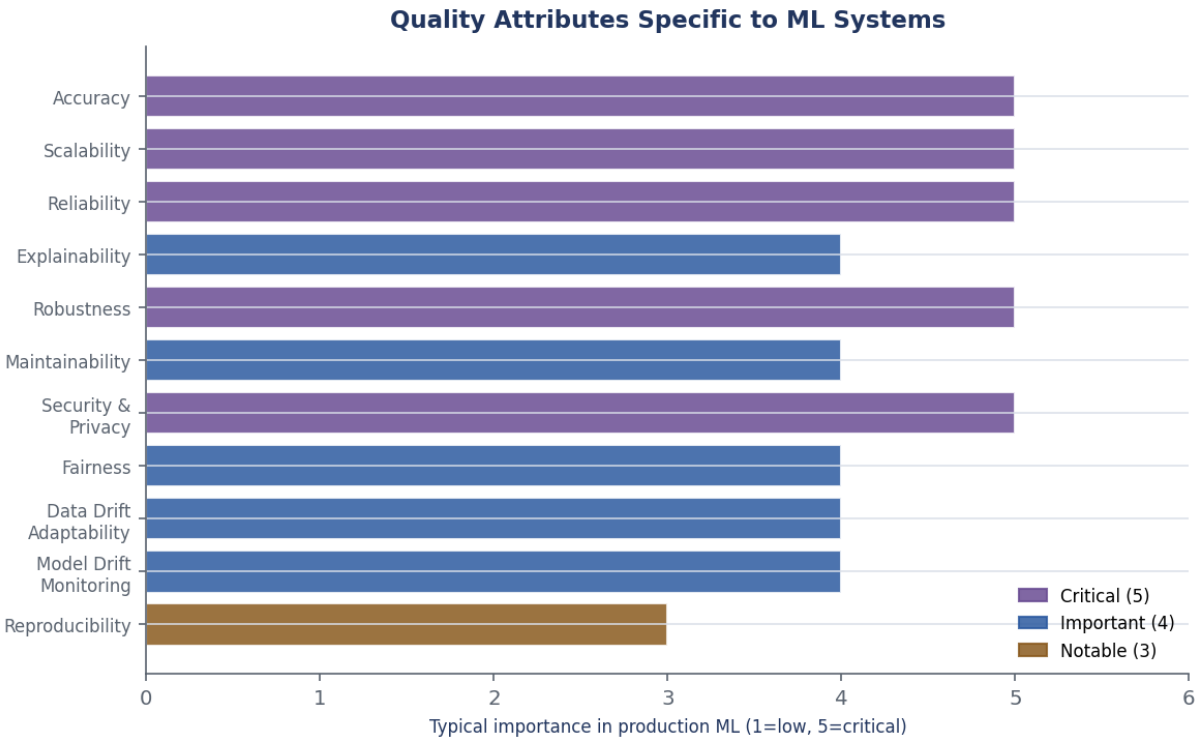


Figure 3: ML-specific quality attributes ranked by typical importance in production. Purple = critical; blue = important; amber = notable. Accuracy, scalability, and reliability top the list, but fairness and robustness are close behind in real-world deployments.

**✗ Watch out**

Do not skip ML-specific QAs and only write the standard software ones. A model that is performant but not reproducible is a maintenance nightmare. A model that is accurate but not fair can cause real harm to users. Both types of QA must appear in your requirements.

✓ **Key takeaway**

ML systems need extra QAs: robustness, explainability, fairness, drift monitoring, and reproducibility. Each one must be SMART — with a target number and a test.

**Where this shows up in ML.** Each ML QA drives a concrete design choice. Explainability ⇒ add a SHAP layer to the serving API. Reproducibility ⇒ fix random seeds and version every dataset. Fairness ⇒ use stratified sampling; add a fairness-aware loss. The QA list is the architect's to-do list.

## 4 What is system architecture?

**Software architecture** (IEEE definition):

> "The software architecture of a program or computing system is the structure or structures of the system, which include software elements, the externally visible properties of those elements, and the relationships among them."

Key words: *structure*, *elements*, and *relationships*. Architecture is the map of what exists and how it connects.

A full **system architecture** covers more than software. It includes five layers working together:

1. **Hardware and network** — servers, GPUs, storage, cables.
2. **Infrastructure** — cloud (AWS, Azure, GCP), containers (Docker), orchestration (Kubernetes).
3. **Software components** — APIs, authentication, business logic services.
4. **ML components** — feature store, model registry, serving engine, monitoring.
5. **Data components** — databases, vector stores, streaming pipelines.

☞ **The intuition**

Architecture is the blueprint before construction. A house blueprint shows rooms, walls, pipes, and wires — not decorations. A system architecture shows components, their connections, and the data that flows between them. The blueprint does not tell builders *how* to mix concrete. It tells them *where* each wall goes and why.

✏ **Worked example: RAG enterprise chatbot architecture**

A company builds an internal HR chatbot. We lay out all five layers:

**Step 1.** **Hardware/infrastructure.** AWS EC2 GPU instances for embedding. ECS Fargate for the API. S3 for document storage.

**Step 2.** **Software components.** API Gateway (rate limiting, auth) feeds an orchestration layer built with LangChain.

**Step 3.** **ML components.** Embedding model (sentence-transformers/all-MiniLM) converts each query to a vector. Retriever fetches the top-5 matching document chunks from Pinecone (a vector database). LLM (GPT-4 via API) generates the final answer. Re-ranking model re-scores the chunks before the LLM sees them.

**Step 4.** **Data components.** HR policy PDFs live in S3. Chunks and embeddings live in Pinecone. Interaction logs go to PostgreSQL.

**Step 5.** **Deployment components.** Docker containers. GitHub Actions CI/CD pipeline. CloudWatch for monitoring.

**Step 6.** **Trace back to QAs.** Accuracy QA ⇒ chose GPT-4 as the LLM. Explainability QA ⇒ every answer cites the source doc. Maintainability QA ⇒ new policy PDFs added to S3 trigger automatic re-indexing — no manual work needed. Security QA ⇒ no data leaves the company's AWS account.

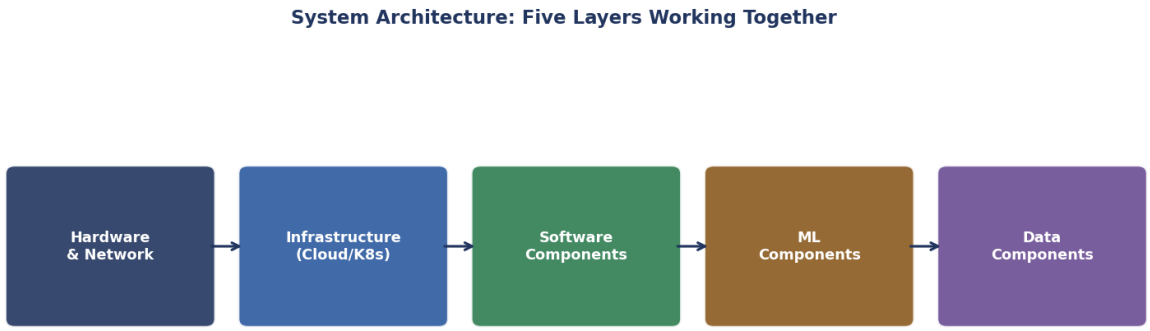


Figure 4: The five layers of a system architecture, from hardware on the left to data on the right. Each layer depends on the one to its left. A QA that touches one layer (e.g. scalability) often forces changes across all layers.

**Where this shows up in ML.** QAs drive architecture at every layer. A low-latency QA means GPU instances at the hardware layer and a quantised model at the ML layer. A security QA means a private VPC at the infrastructure layer and encrypted storage at the data layer. Architecture is the translation of QAs into decisions at each layer.

✓ **Key takeaway**

System architecture maps five layers: hardware, infrastructure, software, ML, and data. Every QA forces a decision at one or more of those layers.

## 5 Architectural patterns

An **architectural pattern** is a reusable solution to a common problem in a given context. We describe it as a triple: {Context, Problem, Solution}.

☞ **The intuition**

A pattern is a proven template. It is not a finished design. A house architect knows patterns like "open-plan kitchen-living" or "split level." They pick the pattern that fits the family size and climate. A software architect knows patterns like Pipe-and-Filter or Microservices. They pick the one that fits the QAs and team size.

★ Everyday picture

A school timetable is a pattern. The context is: many students, many rooms, many teachers, a fixed number of slots. The recurring problem is: how do you avoid clashes? The solution is the timetable. The solution does not come with a specific schedule filled in. It comes with the *structure* for making one. That is what patterns do.

This session covers seven patterns. Four are introduced here. The rest are detailed in later sessions.

| Pattern | Session | Core idea |
|---|---|---|
| Pipe-and-Filter | 4 (this one) | Chain independent processing steps |
| CQRS | 5 | Separate "read" and "write" paths |
| Event-Driven | 5 | Components talk via a message bus |
| Broker | 5 | A central coordinator routes requests |
| Monolithic | 5 | One big deployable unit |
| Microservices | 5 | Many small independent services |
| Layered | 5 | Stack layers, each calling only the one below |

✓ Key takeaway

A pattern is a triple: Context, Problem, Solution. Pick the pattern whose context and problem match your situation. Never force a pattern.

**Where this shows up in ML.** Most ML systems combine multiple patterns. A RAG chatbot uses Pipe-and-Filter (query → embed → retrieve → generate) plus CQRS (reads from the vector store; writes to the log database). Knowing the pattern names lets teams discuss design without confusion.

## 6 Pipe-and-Filter pattern

The **Pipe-and-Filter** pattern organises a system as a chain of independent processing steps.

**Context** A system must process a stream of data through several transformations. Each transformation could be reused in another product.

**Problem** How do you organise steps so each one can be developed, tested, and replaced without touching the others?

**Solution** Break the work into **filters**. Each filter reads from a **pipe**, transforms the data, and writes to the next pipe. Filters never talk to each other directly.

☞ The intuition

Think of a factory assembly line. Station 1 cuts metal. Station 2 welds it. Station 3 paints it. Station 4 inspects it. Each station knows only its own job. You can add a polishing station between paint and inspection without changing any other station. That independence is the whole point.

**✏ Worked example: NLP sentiment analysis pipeline**

Input: the sentence `"I am not happy with the service!"`
We run it through five filters:

**Step 1.** **Filter 1 — Tokenise.** Split the sentence into individual words.

$$[\texttt{"I"}, \texttt{"am"}, \texttt{"not"}, \texttt{"happy"}, \texttt{"with"}, \texttt{"the"}, \texttt{"service"}, \texttt{"!"}]$$

**Step 2.** **Filter 2 — Remove stopwords.** Remove common words ("I", "am", "with", "the"). Keep only content words:

$$[\texttt{"not"}, \texttt{"happy"}, \texttt{"service"}]$$

**Step 3.** **Filter 3 — Vectorise (bag-of-words).** Count each remaining word:

$$\{\texttt{not}{:}1,\ \texttt{happy}{:}1,\ \texttt{service}{:}1\}$$

**Step 4.** **Filter 4 — Model inference.** Pass the vector to a logistic regression model with weights $w_{\text{not}}$, $w_{\text{happy}}$, $w_{\text{service}}$, and bias $w_0$. The **sigmoid** function $\sigma(z) = 1/(1+e^{-z})$ squashes the score into a probability:

$$\hat{y} = \sigma(w_0 + w_{\text{not}} + w_{\text{happy}} + w_{\text{service}}) = 0.89$$

Since $\hat{y} = 0.89 > 0.5$, the class is **NEGATIVE**.

**Step 5.** **Filter 5 — Format output.** Wrap the result in a JSON response:

```
{"text": "I am not happy with the service!",
 "sentiment": "NEGATIVE",
 "confidence": 0.89}
```

**Step 6.** **Check the independence guarantee.** Suppose we want better accuracy. We swap Filter 3 from bag-of-words to TF-IDF. We swap Filter 4 from logistic regression to BERT. Filters 1, 2, and 5 do not change at all. That is the Pipe-and-Filter win.

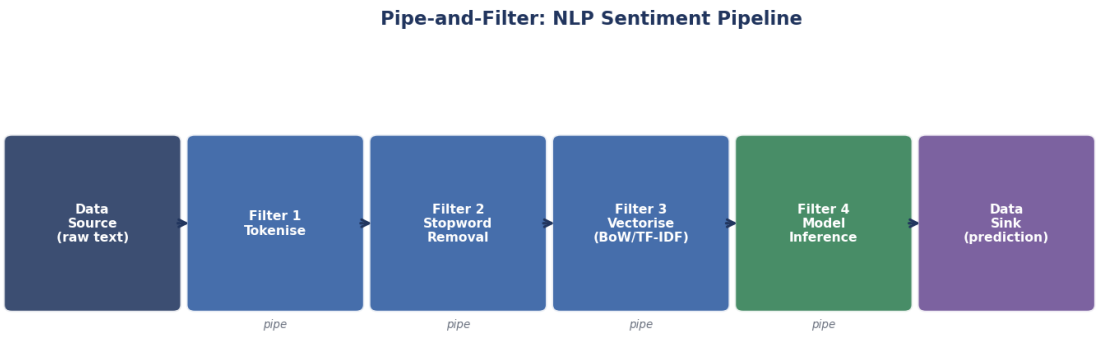


Figure 5: The Pipe-and-Filter NLP pipeline from the worked example. Data flows left to right through five independent filters. Each box (filter) can be replaced without touching its neighbours. The arrows are the pipes that carry data between filters.

**✗ Watch out**

Pipe-and-Filter assumes each filter is *stateless*. A filter that must remember past inputs (e.g. an online learning model) breaks the independence guarantee. Stateful filters need special handling — either store state externally (a database) or switch to an Event-Driven pattern.

**✓ Key takeaway**

Pipe-and-Filter makes ML pipelines modular. Each step — preprocessing, feature extraction, inference, postprocessing — can be tested, versioned, and replaced on its own.

**Where this shows up in ML.** Nearly every ML pipeline is a Pipe-and-Filter in disguise. Apache Spark ML uses it explicitly (transformers and estimators in a `Pipeline` object). MLflow and Kubeflow Pipelines both model each stage as a filter. Knowing the pattern helps you design pipelines that are easy to test and easy to change.

## Glossary & symbols

**Key terms.**

**Quality attribute**
: A measurable property that says *how well* the system must perform — e.g., latency, accuracy, uptime.

**NFR**
: Non-functional requirement. Another name for quality attribute.

**SMART QA**
: A quality attribute that is Specific, Measurable, Attainable, Relevant, and Time-sensitive.

**Software architecture**
: The structure of a system: its components, their properties, and the relationships between them (IEEE definition).

**System architecture**
: Extends software architecture to include hardware, infrastructure, ML, and data components.

**Architectural pattern**
: A reusable solution template for a common design problem. Expressed as {Context, Problem, Solution}.

**Pipe-and-Filter**
: An architectural pattern where data flows through a chain of independent filters connected by pipes.

**Filter**
: One processing step in a Pipe-and-Filter system. Takes input, transforms it, outputs the result. Stateless by design.

**Pipe**
: The channel that carries data from one filter to the next.

**Robustness**
: An ML QA: the model still works well on noisy or incomplete inputs.

**Explainability**
: An ML QA: a human can understand why the model made a prediction.

| | |
|---|---|
| **Fairness** | An ML QA: the model gives equally good results for all groups. |
| **Reproducibility** | An ML QA: the same training run always produces the same model. |
| **Data drift** | When the distribution of input data changes after deployment. |
| **Model drift** | When model performance drops after deployment because the world has changed. |
| **CQRS** | Command Query Responsibility Segregation — a pattern that uses separate paths for reads (queries) and writes (commands). |

**Symbols at a glance.**

| Symbol / abbreviation | Meaning |
|---|---|
| QA | Quality attribute |
| NFR | Non-functional requirement |
| SMART | Specific, Measurable, Attainable, Relevant, Time-sensitive |
| $\hat{y}$ | The model's predicted value (output) |
| $\sigma(z)$ | Sigmoid: $1/(1+e^{-z})$; squashes a score into $(0, 1)$ |
| F1 | Harmonic mean of precision and recall |
| KL divergence | A number measuring how different two distributions are |
| P95 / P99 | The 95th / 99th percentile latency |
| ADAS | Advanced Driver Assistance Systems |
| CDN | Content Delivery Network |
| RAG | Retrieval-Augmented Generation |
| LLM | Large Language Model |
| SHAP | A tool that explains model predictions (SHapley Additive Explanations) |
| CI/CD | Continuous Integration / Continuous Deployment |